# Nexus Interactive Graph Engine

## Documento de visão e especificação técnica

**Versão:** 1.0 · **Data:** 22 de outubro de 2026 · **Idioma:** Português brasileiro
**Status:** Proposta para validação — decisões abertas estão identificadas como hipóteses, recomendação ou alternativa.

---

## 1. Resumo executivo

O **Nexus Interactive Graph Engine** é uma plataforma para transformar conhecimento e ideias em espaços navegáveis de nós e relações. Em vez de manter informações em páginas isoladas, a pessoa cria conceitos, entidades, projetos, eventos e relações tipadas; explora conexões; e, quando fizer sentido, converte parte dessas relações em fluxos executáveis com resultados observáveis.

O projeto nasce da interseção de três interesses: mapas de conhecimento (Nexus), construção de universos conectados (WorldForge) e colaboração interativa (SyncSpace). O produto unifica esses cenários em um motor de grafo genérico, com modelos de domínio configuráveis. Seu diferencial é deixar explícita a distinção entre **uma relação que descreve significado** e **uma aresta que participa de um processo executável**.

A primeira entrega deve provar que um grafo útil pode ser criado, editado, navegado, pesquisado e persistido com uma experiência fluida. Colaboração simultânea, recomendações avançadas, versionamento completo e automações ficam para etapas posteriores, evitando que a complexidade de CRDTs obscureça o valor central.

### Em uma frase

> Um espaço visual e interativo para conectar conhecimento, mundos e processos, tornando as relações compreensíveis e, quando desejado, operáveis.

---

## 2. Visão do produto

### 2.1 Problema

Anotações, documentos e listas guardam conteúdo, mas escondem dependências e conexões. Em projetos de estudo, a pessoa não vê facilmente o que precisa aprender antes de avançar, quais conceitos aparecem em vários projetos ou onde há lacunas. Em worldbuilding, personagens, facções, lugares, eventos e consequências ficam espalhados por documentos e perdem coerência. Ferramentas de diagramação mostram conexões, mas normalmente não oferecem semântica, consulta ou comportamento sobre essas conexões.

### 2.2 Proposta

O Nexus oferece um **grafo vivo** em que:

- cada nó representa um item com tipo e conteúdo próprio;
- cada relação tem um rótulo, direção opcional, propriedades e contexto;
- filtros e vistas ajudam a explorar o mesmo espaço de diferentes maneiras;
- referências entre exemplos de domínio tornam o motor reutilizável;
- uma camada separada pode avaliar fluxos e dependências explicitamente declarados;
- histórico, permissões e colaboração evoluem sem comprometer a integridade do grafo.

### 2.3 Princípios

1. **As relações são conteúdo de primeira classe.** Não são linhas decorativas.
2. **Semântica antes de automação.** Uma conexão só produz efeitos se declarada como fluxo executável.
3. **Exploração sem aprisionamento.** O mapa visual ajuda, mas busca, listas e detalhes também são formas completas de uso.
4. **Modelo genérico, vocabulários específicos.** Um espaço pode usar tipos de estudo, outro tipos de worldbuilding.
5. **Persistência confiável.** A geometria visual não substitui a estrutura canônica dos dados.
6. **Complexidade por fases.** Monólito modular e edição individual antes de distribuição e CRDT.

### 2.4 Público inicial

- pessoas que estudam tecnologia e querem mapear conceitos, pré-requisitos, projetos e trilhas;
- escritores, mestres de RPG e worldbuilders que precisam manter universos internamente coerentes;
- desenvolvedores interessados em grafos, colaboração, eventos e sistemas interativos;
- equipes pequenas que organizam conhecimento ou processos em torno de relações explícitas.

**Hipótese de produto:** começar com uma experiência solo, útil para mapa de conhecimento e worldbuilding, e introduzir colaboração em equipe depois de validar criação, navegação e consulta. A escolha de um único segmento inicial continua aberta.

---

## 3. Conceitos centrais e exemplos

### 3.1 Vocabulário do domínio

- **Workspace:** contêiner de um projeto ou área de trabalho, com membros, papéis e configurações.
- **Graph:** conjunto de nós e relações pertencente a um workspace. O MVP pode tratar cada workspace como um único grafo lógico.
- **Node:** entidade identificável, tipada e versionável, como “TypeScript”, “Reino de Auren” ou “Batalha do Estreito”.
- **Relation:** conexão semântica entre dois nós, com tipo, direção opcional, propriedades e proveniência.
- **View:** configuração de apresentação: posição, zoom, filtros e seleção de nós. Não muda a verdade do domínio.
- **Flow:** definição executável que referencia nós e relações com semântica operacional explícita.
- **Run:** execução imutável de uma versão de flow, com entradas, resultados, falhas e proveniência.

### 3.2 Exemplo: mapa de conhecimento

Nós: **TypeScript**, **JavaScript**, **NestJS**, **HTTP**, **PostgreSQL**, **Prisma**, **API de tarefas**. Relações semânticas: “é construído sobre”, “usa”, “ensina”, “é pré-requisito de”, “aplicado em”. Uma vista de estudo filtra por nível e destaca pré-requisitos. Uma trilha ordena tópicos com base nas relações selecionadas.

Exemplo de distinção: `TypeScript —[é pré-requisito de]→ NestJS` descreve uma dependência pedagógica. Isso não instala pacotes nem dispara uma tarefa. Se houver um fluxo que valide pré-requisitos, ele deve referenciar essa relação e declarar a regra de execução separadamente.

### 3.3 Exemplo: worldbuilding

Nós: **Reino de Auren**, **Casa Veyra**, **Mira**, **Porto de Sal**, **Tratado das Cinzas**, **Guerra do Norte**. Relações: “governa”, “aliado de”, “ocupa”, “participou de”, “causou”, “acontece em”. Propriedades temporais ou narrativas podem contextualizar uma relação (por exemplo, início e fim de uma aliança). Uma timeline ou vista política filtra esse mesmo grafo.

Uma relação `Tratado das Cinzas —[encerrou]→ Guerra do Norte` registra uma afirmação narrativa. Um fluxo de publicação pode reagir à aprovação de um evento, mas só quando uma regra executável for configurada explicitamente.

### 3.4 Relações semânticas versus fluxos executáveis

| **Aspecto** | **Relação semântica** | **Fluxo executável** |
|---|---|---|
| Propósito | Descrever como entidades se relacionam | Definir passos, condições ou efeitos |
| Exemplo | “A depende de B” | “Quando B for concluído, liberar A” |
| Execução | Nunca ocorre por si só | Ocorre via executor e versão definida |
| Alteração | Atualiza conhecimento do grafo | Pode produzir saída ou efeito controlado |
| Risco | Informação imprecisa | Efeitos duplicados, loops, permissões e falhas |
| Auditoria | Autor e histórico da afirmação | Versão, entrada, passos, resultado e idempotência |

Fluxos devem ser **opt-in**, determinísticos quando possível e isolados do CRUD de relações. Não executar código arbitrário fornecido pelo usuário no servidor. Para o MVP, “trilha de aprendizagem” pode ser uma consulta/ordenação explicável, e não um runtime de automação.

---

## 4. Casos de uso prioritários

1. Criar um workspace e escolher um modelo inicial: mapa de conhecimento ou worldbuilding.
2. Criar, editar, tipar, mover e conectar nós no canvas.
3. Abrir um nó para consultar descrição, atributos, relações e atividade recente.
4. Buscar por texto e filtrar por tipo, rótulo, propriedade ou grupo.
5. Navegar relações de um nó e expandir vizinhanças em profundidade limitada.
6. Salvar vistas com filtros e organização visual próprios.
7. Detectar relações quebradas, nós órfãos e ciclos em relações declaradas como dependências.
8. Exportar/importar um workspace em formato JSON documentado.
9. Em fase posterior: compartilhar workspace, editar simultaneamente e ver presença/cursor.
10. Em fase posterior: executar trilhas ou fluxos e inspecionar resultados auditáveis.

**Fora do MVP:** rede social pública, marketplace de grafos, IA generativa como requisito, engine geral de workflow, sincronização offline completa, aplicação móvel nativa e permissões por atributo de cada nó.

## 5. Escopo funcional e roadmap

### Fase 0 — Fundamentos

- validar segmento e vocabulário com protótipos;
- decidir se workspace começa como grafo único ou pode conter múltiplos grafos;
- definir esquemas iniciais de nós e relações;
- testar a navegação de 100–500 nós com React Flow.

### Fase 1 — MVP individual

- cadastro/autenticação básica e workspaces privados;
- CRUD de workspace, nós, relações e vistas;
- canvas com criar, mover, selecionar, conectar e excluir;
- painel de detalhes de nó/relação;
- busca textual básica e filtros por tipo;
- autosave com estado de salvamento explícito;
- undo/redo de sessão ou ações recentes, conforme implementação escolhida;
- modelos iniciais de conhecimento e worldbuilding;
- exportação JSON e documentação de formato;
- logs de auditoria mínimos para alterações persistidas.

### Fase 2 — Exploração e qualidade

- navegação de vizinhança, layout automático opcional, agrupamentos e vistas salvas;
- trilhas de conhecimento explicáveis, detecção de órfãos e ciclos;
- histórico de versões/atividade com restauração controlada;
- convites e permissões por workspace;
- melhor busca, paginação e otimização de consultas;
- métricas de uso e telemetria com privacidade apropriada.

### Fase 3 — Colaboração em tempo real

- presença, cursores e seleção compartilhada;
- sincronização de conteúdo colaborativo delimitado (por exemplo, descrição rica ou quadro);
- resolução de conflitos e persistência/reconexão;
- avaliar Yjs e transporte Socket.IO ou WebSocket dedicado;
- políticas de autorização por documento e workspace.

### Fase 4 — Fluxos e ecossistema

- esquema de flow versionado e executor limitado;
- simulação/validação antes de executar efeitos;

- runs idempotentes, logs e reexecução segura;
- API pública, integrações e importadores adicionais;
- recomendações ou assistências inteligentes, apenas após evidência de valor.

**Roadmap é hipótese de sequência, não compromisso de prazo.** A ordem deve mudar conforme validação de usuário, desempenho e capacidade.

---

# 6. Requisitos

## 6.1 Requisitos funcionais (RF)

- **RF-01:** criar workspaces privados e renomeá-los.
- **RF-02:** criar, editar e arquivar nós com tipo e atributos validados.
- **RF-03:** criar, editar e remover relações com origem, destino e tipo válidos.
- **RF-04:** impedir referências a nós inexistentes e exclusões que deixem relações órfãs.
- **RF-05:** salvar e recuperar posições e configurações de vistas.
- **RF-06:** pesquisar nós por título e descrição e filtrar por tipo.
- **RF-07:** validar permissões em cada operação de leitura e escrita.
- **RF-08:** registrar ator, data, tipo e entidade das alterações relevantes.
- **RF-09:** exportar e importar dados em JSON versionado, com validação antes de aplicar.
- **RF-10:** representar presença e alterações concorrentes na fase colaborativa.
- **RF-11:** diferenciar relações semânticas de definições e execuções de flows.
- **RF-12:** exibir erros de validação, conflito e rede sem perder silenciosamente o trabalho.

## 6.2 Requisitos não funcionais (RNF)

- **RNF-01 Usabilidade:** ações essenciais devem estar disponíveis por canvas e por controles acessíveis.
- **RNF-02 Consistência:** transações devem manter integridade entre nós, relações e eventos.
- **RNF-03 Desempenho:** hipótese inicial: abrir um grafo de até 500 nós com interação básica fluida em máquina comum; definir orçamento mensurável após protótipo.
- **RNF-04 Escalabilidade:** consultas devem limitar profundidade, paginação e tamanho de payload; carregar subgrafos sob demanda.
- **RNF-05 Segurança:** autorização no servidor, isolamento entre workspaces, validação de entrada e proteção contra abuso.
- **RNF-06 Observabilidade:** logs estruturados, correlação de requisições, métricas de latência e erros, sem registrar conteúdo sensível sem necessidade.
- **RNF-07 Recuperação:** backups testáveis e recuperação de salvamento após falha de rede.
- **RNF-08 Portabilidade:** dados exportáveis em formato documentado e independente da camada visual.
- **RNF-09 Acessibilidade:** teclado, foco visível, contraste, rótulos e alternativa textual ao canvas.
- **RNF-10 Manutenibilidade:** módulos com fronteiras claras, migrações versionadas e tipos compartilhados gerados ou publicados de modo controlado.

Metas quantitativas de disponibilidade, RPO/RTO, latência e tamanho máximo de grafo dependem de hospedagem e público; continuam abertas.

---

# 7. UX e interação

## 7.1 Estrutura de tela proposta

- **Barra superior:** workspace, busca, desfazer/refazer, status de salvamento, compartilhar (quando habilitado).
- **Canvas central:** nós e relações, zoom/pan, seleção múltipla, minimapa opcional.
- **Painel lateral:** detalhes e edição do nó ou relação selecionada.
- **Barra lateral de navegação:** vistas, tipos, filtros e favoritos.
- **Estado vazio:** convite para criar primeiro nó, importar exemplo ou escolher modelo.

## 7.2 Interações

- criar nó pelo botão ou atalho; conectar por gesto e escolher rótulo da relação;
- editar propriedades em formulário tipado, sem depender de edição inline minúscula no canvas;
- destacar vizinhança e reduzir ruído por filtros;
- confirmar exclusões relevantes e explicar consequências, especialmente relações incidentes;
- indicar salvamento pendente, salvo, falha e tentativa de reconexão;
- oferecer listas e leitores de relações como alternativa acessível ao canvas;
- não representar fluxo executável com o mesmo estilo visual de uma relação descritiva.

## 7.3 Decisões de UX ainda abertas

- editor rico ou Markdown para descrição;
- auto layout no MVP ou apenas posicionamento manual;
- múltiplas vistas por workspace desde o início;
- comportamento de exclusão: cascata, arquivamento ou bloqueio quando há relações;
- suporte a dispositivos pequenos e definição de limites para canvas móvel.

---

# 8. Stack proposta

| Camada | Recomendação inicial | Alternativas / observações |
|---|---|---|
| Linguagem | TypeScript ponta a ponta | Ruby on Rails permanece alternativa válida; NestJS foi a preferência de exploração mencionada |
| API | NestJS em monólito modular | REST para CRUD; GraphQL só se consultas compostas justificarem custo |

| Camada | Recomendação inicial | Alternativas / observações |
|---|---|---|
| Frontend | Next.js + React | Renderização do canvas predominantemente no cliente; separar contratos de domínio da UI |
| Canvas | React Flow | Comparar limites de performance e acessibilidade com Cytoscape.js se a escala crescer |
| Persistência | PostgreSQL | JSONB para atributos flexíveis com campos centrais normalizados |
| ORM/migração | Prisma | SQL explícito para consultas recursivas, índices especializados ou casos não cobertos pelo ORM |
| Tempo real inicial | Nenhum no MVP | Não adicionar transporte antes de validar colaboração |
| Colaboração futura | Yjs + Socket.IO ou WebSocket dedicado | Yjs resolve sincronização de estruturas compartilhadas; Socket.IO é transporte/presença, não substitui modelo de autorização ou persistência |
| Cache/fila | Nenhum no começo; Redis/BullMQ quando necessário | Para jobs, fan-out ou throttling comprovado |
| Testes | Vitest/Jest, Supertest, Playwright | Selecionar um runner de unidade e padronizar; não duplicar ferramentas sem motivo |
| Infraestrutura | Docker, CI (GitHub Actions ou equivalente), deploy gerenciado | Provedor e estratégia de hospedagem permanecem em aberto |

**Recomendação:** NestJS + Next.js + PostgreSQL/Prisma formam uma stack consistente com o objetivo de aprendizado full-stack. Confirmar compatibilidade de versões e dependências no início da implementação; este documento não fixa versões de pacote.

---

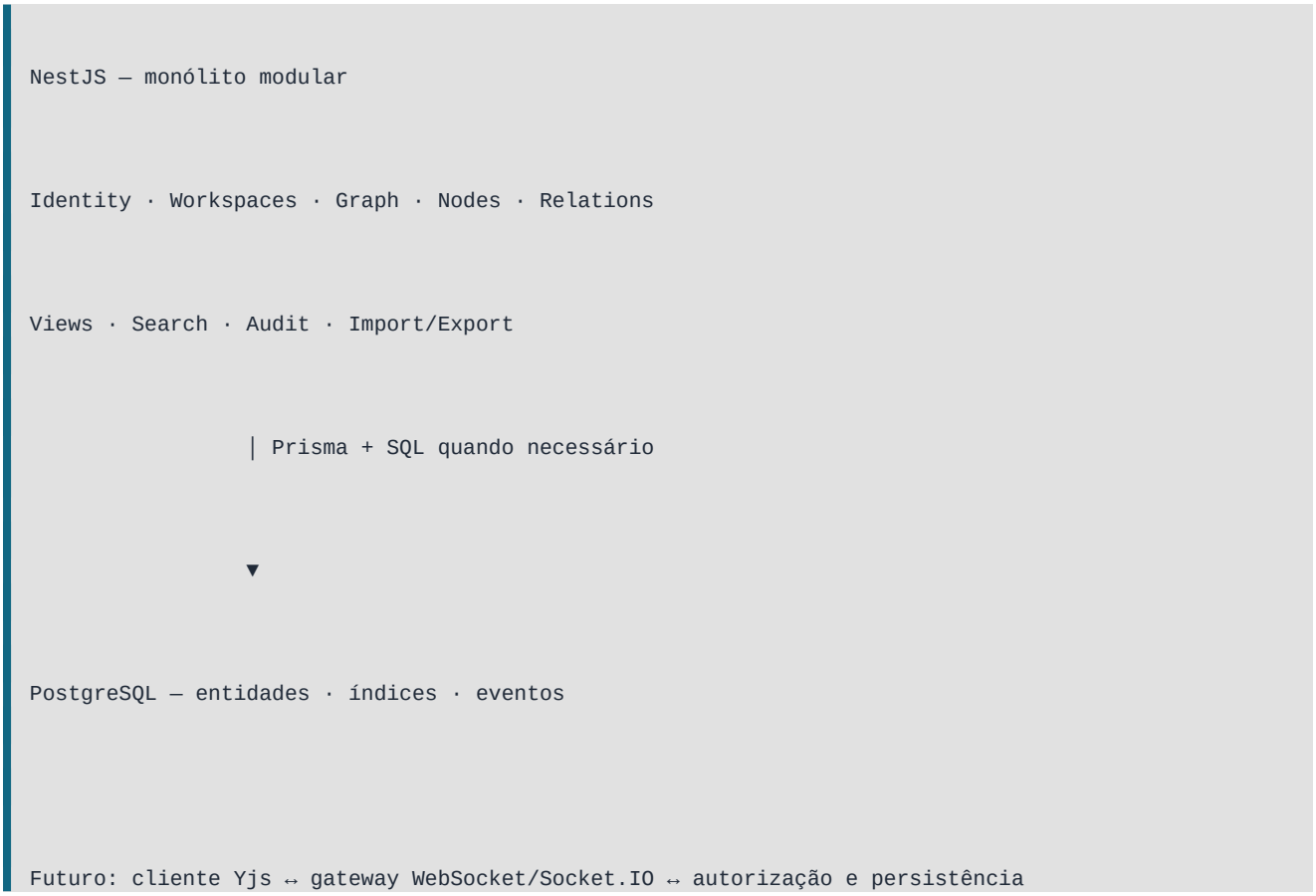
## 9. Arquitetura lógica

```
Next.js + React


Canvas React Flow · busca · propriedades · vistas


                | HTTPS / REST no MVP


                ▼
```




## 9.1 Módulos do backend

- **Identity:** autenticação, sessão/token e identidade do ator.
- **Workspaces:** criação, membros, papéis e políticas.
- **Graph:** invariantes do grafo e operações de domínio.
- **Nodes / Relations:** validação, comandos e consultas específicas.
- **Views:** posições, zoom, filtros e preferências de apresentação.
- **Search:** busca textual e filtros; pode começar dentro de Graph.
- **Audit:** registro de alterações e consulta de atividade.
- **ImportExport:** validação de formato, limites, importação transacional.
- **Collaboration (futuro):** presença, sincronização e recuperação de estado colaborativo.
- **Flows (futuro):** definições, validação, execução e registro de runs isolados.

Evitar microserviços no começo: o domínio ainda está em descoberta e transações entre entidades são importantes. Separar processos apenas quando escala, isolamento de falhas ou times justificarem.

## 9.2 Fluxo típico de escrita

1. cliente envia comando autenticado, por exemplo `CreateRelation`;
2. controller valida formato e encaminha ao caso de uso;
3. autorização confirma acesso ao workspace;
4. domínio valida tipos, limites e invariantes;
5. uma transação grava relação e evento de auditoria;

6. resposta retorna entidade canônica e versão/updatedAt;
7. cliente atualiza o canvas a partir da resposta.

---

## 10. Modelo de dados inicial

Esquema conceitual — nomes e cardinalidades podem mudar após protótipo:

```
User(id, email, displayName, createdAt)

Workspace(id, name, ownerId, createdAt, updatedAt)

Membership(workspaceId, userId, role, createdAt)

Graph(id, workspaceId, name, schemaVersion, createdAt)

Node(id, graphId, typeKey, title, content, attributesJson,

     createdBy, createdAt, updatedAt, archivedAt?)

Relation(id, graphId, sourceNodeId, targetNodeId, typeKey,

         directed, propertiesJson, createdBy, createdAt, updatedAt)

GraphView(id, graphId, ownerId?, name, viewportJson, filtersJson,

          createdAt, updatedAt)

AuditEvent(id, workspaceId, actorId, entityType, entityId,

          action, payloadJson, createdAt, requestId)

FlowDefinition(id, graphId, name, definitionJson, version, status)  [futuro]

FlowRun(id, flowDefinitionId, version, inputJson, status,
```

```
        resultJson, startedAt, finishedAt, idempotencyKey)          [futuro]
```

### 10.1 Regras e índices sugeridos

- IDs UUID/ULID: escolher um padrão; UUID é recomendação inicial simples.
- FKs de `Relation` para nós devem impedir referência órfã; decidir política de exclusão conscientemente.
- índices em `graphId`, `(graphId, typeKey)`, `(sourceNodeId)`, `(targetNodeId)` e campos de busca relevantes;
- unicidade ou não de relações paralelas deve ser decisão de domínio: por padrão permitir relações paralelas se tipo/propriedades diferirem;
- JSONB para atributos flexíveis, com validação por schema versionado na aplicação;
- posições podem ficar em `GraphView` (por usuário/vista), separadas do conteúdo semântico;
- não guardar todo o grafo como um único blob: dificulta integridade, busca e escrita concorrente.

### 10.2 Eventos de domínio/auditoria

Exemplos: `WorkspaceCreated`, `NodeCreated`, `NodeUpdated`, `NodeArchived`, `RelationCreated`, `RelationUpdated`, `RelationDeleted`, `ViewUpdated`, `ImportCompleted`. Cada evento inclui identificador, ator, timestamp, workspace, entidade, versão ou request ID quando aplicável. O MVP pode usar uma tabela de auditoria transacional; isso **não** obriga event sourcing. Outbox só se efeitos assíncronos confiáveis forem introduzidos.

---

## 11. Propagação e consulta em grafos

### 11.1 O que significa “propagação” aqui

Propagação pode significar coisas diferentes e não deve virar um mecanismo implícito:

1. **Exploração visual:** buscar vizinhos a uma profundidade escolhida.
2. **Propagação de dependências:** inferir o que é bloqueado ou habilitado por pré-requisitos.
3. **Efeito operacional:** disparar ações em um flow.

O MVP implementa exploração e, opcionalmente, análise de dependências como consultas explicáveis. Efeitos operacionais ficam fora do CRUD e exigem flows.

### 11.2 Consulta de vizinhança

- começar por vizinhos de 1 salto e permitir expansão progressiva;
- impor limite de profundidade, quantidade de nós e tempo de consulta;
- escolher se a direção é respeitada, ignorada ou controlada pelo usuário;
- devolver relações junto com nós para preservar contexto;
- paginar ou carregar sob demanda em grafos grandes;
- usar CTE recursiva do PostgreSQL ou consultas em camadas; avaliar benchmarks antes de introduzir banco de grafo.

### 11.3 Ciclos e ordenação

Ciclos são válidos em muitos domínios (alianças, influência, referências). Só relações classificadas como `prerequisite` ou equivalente devem ser tratadas como DAG quando o caso de uso exigir. Verificação pode ocorrer ao criar uma aresta desse tipo; ordenar trilhas usa ordenação topológica sobre o subgrafo selecionado. Nunca inferir uma regra universal de aciclicidade para todo o grafo.

### 11.4 Banco relacional ou banco de grafos

**Recomendação inicial:** PostgreSQL, por simplicidade operacional, integridade e adequação a grafos moderados com consultas de vizinhança limitadas. Reavaliar Neo4j ou outra solução quando consultas de caminhos complexos, escala ou latência forem demonstradamente inadequadas. A troca não deve ser guiada apenas pela aparência do produto como “grafo”.

---

## 12. Colaboração e consistência

### MVP

- edição individual com autosave, controle de versão otimista (`updatedAt` ou `version`) e tratamento de conflito;
- API retorna conflito legível se a versão enviada estiver desatualizada;
- atualizações do canvas seguem resposta canônica do servidor.

### Fase colaborativa

- primeiro definir quais dados exigem edição simultânea (texto, canvas, ou ambos);
- Yjs para estado compartilhado com CRDT e rebase de mudanças concorrentes;
- Socket.IO pode fornecer transporte, salas e presença; presença não é fonte canônica de dados;
- avaliar persistência de atualizações Yjs, snapshots, compactação e reconexão;
- impor autorização antes de entrar em sala e em cada operação relevante;
- evitar misturar transações relacionais e atualizações CRDT sem uma fronteira explícita;
- testar corrida de exclusão/edição, reconexão, duplicação e estado parcialmente persistido.

**Decisão em aberto:** usar Yjs apenas em documentos ricos/descrições ou também no grafo completo. Recomendação: começar por uma área delimitada; manter nós e relações canônicos no PostgreSQL até haver necessidade comprovada de sincronização CRDT do grafo inteiro.

---

## 13. Segurança e privacidade

- autenticação robusta; escolher provedor próprio ou serviço gerenciado;
- autorização por workspace e papel (`owner`, `editor`, `viewer` como ponto de partida);
- validar acesso a cada recurso, inclusive em WebSocket e exportações;
- validar payloads com schemas e limites de tamanho/profundidade;
- parametrizar SQL e restringir consultas recursivas por limite;
- aplicar rate limiting e proteção contra importações abusivas;
- escapar/limitar conteúdo renderizado para evitar XSS; sanitizar HTML se houver rich text;

- armazenar segredos fora do repositório e rotacionar credenciais;
- definir retenção de auditoria e exclusão de conta/workspace;
- não executar JavaScript arbitrário nos flows; oferecer catálogo de passos seguros e sandbox no futuro;
- documentar backup, restauração e política de logs sem conteúdo sensível.

A jurisdição, requisitos legais aplicáveis e provedor de hospedagem precisam ser definidos antes de uso com dados pessoais ou de equipe.

---

## 14. Testes e qualidade

- **Unidade:** invariantes de domínio, validação de relações, autorização e análise de ciclos.
- **Integração:** repositórios Prisma/PostgreSQL, transações e migrações.
- **API:** autenticação, isolamento entre workspaces, erros e limites.
- **UI:** criação/edição/conexão, estado vazio, filtros e falha de salvamento.
- **E2E:** fluxo principal desde criar workspace até exportar/importar.
- **Concorrência (fase futura):** duas sessões editando, perda/reconexão e operações simultâneas.
- **Desempenho:** cenários sintéticos de 100, 500 e 2.000 nós para definir limites reais, em vez de prometer escala sem medição.
- **Acessibilidade:** navegação por teclado, leitura dos controles e alternativa em lista.

Não exigir cobertura percentual como objetivo isolado. Priorizar riscos e regras de domínio.

---

## 15. CI/CD e operação

Pipeline recomendado:

1. instalar dependências de forma reproduzível e validar lockfile;
2. lint e formatação;
3. verificação de tipos;
4. testes de unidade e integração;
5. build de frontend e backend;
6. análise de dependências e segredos;
7. aplicar migrações em ambiente de staging;
8. deploy com health checks e rollback documentado.

Ambientes: desenvolvimento local com Docker Compose para PostgreSQL; staging; produção. Segredos via serviço próprio do provedor. Backups automatizados e teste periódico de restauração. Observabilidade mínima com logs estruturados, métricas e alertas para erros/latência. Provedor, domínio, estratégia de deploy e orçamento ainda não foram escolhidos.

---

## 16. Decisões técnicas registradas

| Decisão proposta | Motivo | Estado |
|---|---|---|
| NestJS + TypeScript para API | Alinha-se à preferência inicial e permite módulos e contratos tipados | Recomendada, não confirmada |
| Next.js + React para UI | Ecossistema e integração com React Flow | Recomendada, não confirmada |
| PostgreSQL como fonte de verdade | Integridade, transações e consultas adequadas ao início | Recomendada, reavaliável |
| Prisma como acesso principal | Produtividade e migrações tipadas | Recomendada, SQL explícito permitido para casos especiais |
| Monólito modular | Mantém simplicidade durante descoberta e transações | Recomendada |
| Sem tempo real no MVP | Valida valor central antes do custo de sincronização | Recomendação de escopo |
| Yjs em fase posterior | Resolve conflitos de edição compartilhada onde CRDT fizer sentido | Hipótese de tecnologia |
| Socket.IO como opção de transporte | Facilita salas/presença; não define persistência nem autorização | Alternativa a validar |
| Fluxos separados de relações | Evita efeitos implícitos e reduz risco | Princípio recomendado |
| Um workspace/um grafo no MVP | Simplifica navegação e modelo inicial | Hipótese, revisar na Fase 0 |

### Alternativas relevantes

- **Rails + Next.js:** opção válida com produtividade alta; NestJS favorece consistência TypeScript e aprofundamento desejado.
- **GraphQL:** pode ajudar em consultas compostas do canvas, mas REST reduz custo inicial e é suficiente para os casos previstos.
- **Banco de grafos:** considerar quando consultas e escala justificarem; não é pré-requisito conceitual.
- **CRDT completo desde o início:** oferece colaboração avançada, mas aumenta complexidade de persistência, autorização, depuração e UX.

---

## 17. Riscos e mitigação

| Risco | Efeito | Mitigação |
|---|---|---|
| Escopo amplo (wiki + grafo + colaboração + automação) | MVP demora e não valida produto | Fatiar por fases; priorizar criar, conectar, explorar e salvar |

| Risco | Efeito | Mitigação |
|---|---|---|
| Canvas vira único modo de interação | Acessibilidade e escala ruins | Busca, listas, painel de detalhes e carregamento por subgrafo |
| Relações sem vocabulário consistente | Grafo difícil de consultar | Tipos configuráveis com rótulos e orientação documentados |
| Modelagem JSON excessiva | Validação e busca frágeis | Campos centrais normalizados e schemas versionados |
| Consultas explosivas | Lentidão ou abuso | Limites de profundidade, paginação, índices e orçamento de consulta |
| Tempo real prematuro | Bugs de conflito e persistência | Adiar; delimitar primeiro o conteúdo colaborativo |
| Efeitos implícitos de relações | Surpresas e ações inseguras | Fluxos explícitos, versionados e auditados |
| Crescimento de dependência/provedor | Custo e lock-in | Exportação aberta, contratos próprios e revisão periódica |
| Falta de foco de público | UX genérica demais | Validar um segmento inicial e modelos concretos |

---

## 18. Critérios de aceite do MVP

O MVP estará pronto para validação quando:

1. uma pessoa puder criar um workspace privado e escolher um modelo inicial;
2. puder criar, editar, mover e arquivar nós sem perder dados;
3. puder conectar nós por relações tipadas e consultar seus detalhes;
4. puder pesquisar, filtrar e navegar vizinhos com limites claros;
5. posições e vistas persistirem independentemente do conteúdo semântico;
6. atualização conflitante ou erro de rede for mostrado sem perda silenciosa;
7. nenhum usuário puder ler ou alterar workspace alheio por manipulação de IDs;
8. exclusão/importação preservarem integridade e tiverem consequência compreensível;
9. exportação JSON puder ser validada e reimportada em workspace de teste;
10. a interface oferecer alternativa textual/acessível para inspecionar o grafo;
11. testes cobrirem regras centrais, isolamento e fluxo principal;
12. documentação explicar que relações semânticas não executam ações.

Os limites numéricos de desempenho devem ser definidos após protótipo e registrados antes de considerar o requisito atendido.

---

## 19. Perguntas ainda abertas

- Qual é o primeiro público para validação: estudantes/desenvolvedores ou worldbuilders?
- Workspace contém exatamente um grafo ou múltiplos grafos desde o MVP?
- O MVP precisa de autenticação real ou pode começar com ambiente local e usuário único?
- Descrições serão Markdown, texto simples ou editor rico?
- Tipos de nó/relação serão globais, por template ou configuráveis por workspace?
- Como versionar/restaurar alterações e qual retenção de histórico é necessária?
- Qual hospedagem, orçamento e requisitos de privacidade são esperados?
- Colaboração deve priorizar texto, canvas, ou os dois?
- “Propagação” do grafo significa principalmente trilhas, impacto/dependências, ou automações?

---

## 20. Próximos passos recomendados

1. escolher um segmento inicial e escrever três jornadas de uso reais;
2. prototipar canvas com 100–500 nós e testar navegação, filtros e alternativa em lista;
3. validar o vocabulário e o modelo mínimo de `Node`, `Relation` e `View`;
4. implementar um vertical slice: workspace → nós → relações → persistência → busca;
5. medir fricções e desempenho antes de expandir o escopo;
6. só então priorizar histórico avançado, colaboração ou flows.

---

## Apêndice A — Exemplo de formato de exportação

```
{

  "format": "nexus-graph",

  "schemaVersion": 1,

  "workspace": { "name": "Mapa de estudos" },

  "graph": { "name": "Tecnologia" },

  "nodes": [

    { "id": "uuid", "type": "concept", "title": "TypeScript", "content": "...", "attributes": {} }
```

```
  ],

  "relations": [

    { "id": "uuid", "source": "uuid", "target": "uuid", "type": "prerequisite", "directed": true,
"properties": {} }

  ],

  "views": []

}
```

Importadores devem validar versão, limite de tamanho, referências e schemas antes de gravar. IDs externos podem ser remapeados, preservando referências dentro do arquivo.

## Apêndice B — Exemplo conceitual de evento

```
{

  "id": "uuid",

  "type": "RelationCreated",

  "workspaceId": "uuid",

  "actorId": "uuid",

  "entityId": "uuid",

  "occurredAt": "2026-10-22T12:00:00Z",

  "requestId": "uuid",

  "payload": { "sourceNodeId": "uuid", "targetNodeId": "uuid", "typeKey": "prerequisite" }
```

```
}
```

Evento de auditoria explica uma mudança já confirmada; não é, por si só, um comando para executar efeitos externos.